PolicyPal — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and the shape of the graph

Three roles on a LangGraph graph, plus a RAG layer:

- `src/rag/` — `chunking.py` (paragraph chunks), `store.py` (`get_collection`, embedding function as a parameter), `ingest.py` (`build_vector_store` — adds chunks with a `{"source": file}` metadata), `retriever.py` (`retrieve` → list of `{text, source, distance}`).
- `src/agents/retriever_agent.py` — `classify_kind` (question vs action, **in code**), `search` (retrieve + drop anything above `MAX_DISTANCE`).
- `src/agents/reasoner_agent.py` — `answer_grounded` (from passages only, cited; `NOT_FOUND` when empty).
- `src/agents/approval_agent.py` — `build_checklist`.
- `src/state.py` — `DocState`; `src/graph.py` — `build_graph(model, collection, checkpointer)`.

Graph shape: `START → retrieve`; `retrieve` returns `Command(goto="reason" or "approval")`; `reason → END`; `approval` pauses, then `→ END`.

Hint 2 — retrieval, the distance cutoff, and "not found"

Chroma's distance is **squared-L2: lower = closer**. `retrieve` returns each hit's `distance` and its source file. The "not found" behaviour is just a cutoff:

```
MAX_DISTANCE = 1.2      # calibrated for Chroma's default embeddings
def search(collection, question):
    return [p for p in retrieve(collection, question)
            if p["distance"] <= MAX_DISTANCE]   # (write as a plain loop)
```

Anything above the cutoff isn't relevant, so an off-topic question yields an **empty** list — which the reasoner turns into the honest refusal. Store each chunk's source file in its metadata at ingest time, so the answer can cite it.

For the unit test, use a tiny offline keyword embedding so there's no model download — but **L2-normalize its vectors**, so its distances land in the same 0–2 range the `1.2` cutoff was tuned for. (Raw word counts give huge distances and everything looks "not relevant".)

Hint 3 — grounded + cited, and refusing cleanly

The reasoner answers from the passages only, and refuses *without* a model call when there's nothing to ground on:

```
NOT_FOUND = "That isn't covered in the HR policies."
def answer_grounded(model, question, passages):
    if len(passages) == 0:
        return NOT_FOUND
    lines = [f"[{p['source']}] {p['text']}" for p in passages]  # plain loop
    chain = prompt | model | StrOutputParser()
    return chain.invoke({"question": question, "passages": "\n".join(lines)})
```

Put the citation rule in the prompt (*"cite the source file like (leave_policy.md)"*) and the "say exactly this if not present" line, so a covered answer is grounded and cited and an edge the retriever missed still refuses rather than invents.

Hint 4 — the approval pause, and resuming from another run

The approval node pauses the whole graph with `interrupt()` and waits:

```
def approval_node(state):
    decision = interrupt({"checklist": build_checklist(state["question"])})
    if decision == "yes":
        done = f"Recorded: {state['question']}"
        return Command(goto=END,
                       update={"approved": True, "result": done})
    return Command(goto=END,
                   update={"approved": False, "result": "Action cancelled."})
```

Two things make it real:

- **Compile with a checkpointer** (`MemorySaver` in-process; `SqliteSaver.from_conn_string(path)` to survive across processes) and run with a `thread_id` . After the first `invoke` , `app.get_state(cfg).next` is non-empty — that's the pause. Resume with `app.invoke(Command(resume="yes"), cfg)` .
- Because the state is saved under the `thread_id` , `resume_demo.py` can open the **same** SQLite file in a brand-new process and finish the run — that's the whole point of saving state

instead of blocking.

Still stuck? The whole thing, every file, is in the complete solution.